Apuntes PSP – Procesos

ndice

Índice rápido

Busca con Ctrl+F la palabra clave entre corchetes (por ejemplo `[PB]`) para saltar directo a la sección y copiar su código.

Clave	Qué necesitas hacer	Sección
`[TEORIA]`	Repasar conceptos (estados, BCP, algoritmos…)	1. Teoría
`[PB]`	Lanzar un proceso con ProcessBuilder y leer lo que imprime	2. ProcessBuilder
`[RT]`	Lanzar un proceso con Runtime.exec	3. Runtime
`[WAIT]`	Esperar a que termine el hijo / código de salida	4. waitFor
`[STDIN]`	Mandar datos del padre al hijo (escribir en su teclado)	5. Entrada estándar
`[REDIR]`	Redirigir entrada, salida o errores a ficheros	6. Redirecciones
`[METODOS]`	isAlive, destroy, getErrorStream, info del proceso	7. Otros métodos
`[PLANTILLA]`	Plantilla completa y errores típicos de examen	8. Plantilla

Regla general: el padre es la clase con `main` que usa `ProcessBuilder`/`Runtime`; el hijo es otro programa (otra clase Java compilada, un `.py`, un comando como `ls`). El hijo NO comparte variables con el padre: solo se comunican por streams (entrada/salida), ficheros o el código de salida.

1. Teoría [TEORIA]

Programa vs proceso

- **Programa:** código fuente o ejecutable guardado en disco. Es algo estático.
- **Proceso:** programa en ejecución. Tiene memoria propia, PID, estado y recursos asignados por el SO.

- **Ejecutado vs interpretado:** un programa compilado (C) se ejecuta directamente; uno interpretado (Python, Java bytecode) lo ejecuta un intérprete. En `ps -ef` se ve el ejecutable (`./programa`) o el intérprete (`java Clase`, `python3 script.py`).
- **Servicio / demonio:** proceso que corre en segundo plano sin interfaz, normalmente arrancado por el sistema (ej. `sshd`, `cron`).

Hardware y tipos de programación

- **Monoprocesador / multiprocesador** y **uno / varios núcleos:** con un solo núcleo la concurrencia es intercalada; con varios puede ser paralelismo real.
- **Monotarea:** un proceso cada vez. **Multitarea:** varios procesos a la vez (alternando o en paralelo).
- **Programación concurrente:** varias tareas progresan a la vez, aunque sea turnándose en un núcleo.
- **Programación paralela:** varias tareas se ejecutan físicamente a la vez en varios núcleos/procesadores.
- **Programación distribuida:** las tareas corren en varias máquinas conectadas por red.

Contexto de un proceso y BCP

El **Bloque de Control de Proceso (BCP / PCB)** es la estructura donde el SO guarda todo lo necesario para pausar y reanudar un proceso:

- **PID:** identificador del proceso.
- **Contador de programa:** dirección de la siguiente instrucción.
- **Imagen de memoria:** código, datos y heap del proceso.
- **Pila de ejecución:** llamadas a funciones y variables locales.
- **Estado del proceso**, registros de CPU, ficheros abiertos, prioridad.

Estados de un proceso

1. **Nuevo:** se está creando.
2. **Listo:** preparado, esperando que le den CPU.
3. **En ejecución:** usando la CPU.
4. **Bloqueado:** esperando un evento (E/S, recurso). Al llegar el evento vuelve a Listo.
5. **Finalizado:** ha terminado.

Transiciones: Nuevo → Listo → Ejecución → (Bloqueado → Listo) o (Listo, si le quitan la CPU) o Finalizado.

Planificador

- **Largo plazo:** decide qué procesos nuevos entran al sistema (pasan a Listo).
- **Corto plazo:** decide qué proceso Listo pasa a Ejecución.
- **Cambio de contexto:** el SO guarda el BCP del proceso que sale (registros, contador de programa, estado) y carga el BCP del que entra. Es tiempo "perdido" sin trabajo útil.

Tipos de planificación a corto plazo

- **Sin desalojo / cooperativa:** el proceso suelta la CPU solo cuando termina o se bloquea.

- **Apropiativa (con desalojo):** el SO puede quitarle la CPU (por ejemplo si llega uno de más prioridad).
- **Tiempo compartido:** cada proceso recibe un trozo de tiempo (quantum) y luego cede el turno.

Algoritmos de planificación

Algoritmo	Cómo elige	Desalojo
FCFS (First Come First Served)	El primero que llega	Normalmente no apropiativo
SJF (Shortest Job First)	El de menor tiempo de CPU	Sin desalojo o apropiativo (SRTF)
RR (Round Robin)	Turnos con quantum fijo	Apropiativo
Por prioridad	El de mayor prioridad	Apropiativo (riesgo de inanición)

Comunicación entre procesos

- **Sockets:** comunicación por red (o local).
- **Flujos de entrada/salida (pipes):** el padre lee la salida del hijo o escribe en su entrada (lo que hacemos con `getInputStream` / `getOutputStream`).
- **RPC (Remote Procedure Call):** llamar a una función de otro proceso como si fuera local.
- **Persistencia:** ficheros, bases de datos compartidas.

Recursos compartidos y sincronización

- **Sección crítica:** trozo de código que accede a un recurso compartido.
- **Exclusión mutua:** solo uno a la vez dentro de la sección crítica.
- **Sincronización:** coordinar el orden en que actúan procesos/hilos.
- **Operación atómica:** se ejecuta entera sin que nadie pueda interrumpirla a medias.
- **Interbloqueo (deadlock):** dos o más se esperan mutuamente y ninguno avanza.
- **Inanición (starvation):** uno nunca consigue el recurso porque siempre se lo llevan otros.

2. Lanzar proceso con ProcessBuilder y leer su salida [PB]

Pasos: (1) crear el `ProcessBuilder` con el comando y sus argumentos, (2) opcional `directory()` para la carpeta donde se ejecuta, (3) `start()` devuelve un `Process`, (4) leer `getInputStream()` = lo que el hijo imprime por pantalla.

Padre

```
import java.io.BufferedReader;
import java.io.File;
import java.io.IOException;
import java.io.InputStreamReader;

public class Padre {
```

```
    public static void main(String[] args) {
        // Comando + argumentos (cada argumento en una posicion)
        String[] sArg = new String[2];
        sArg[0] = "java";          // o ruta completa: /usr/java/jdk-
21.0.4/bin/java (ver con: which java)
        sArg[1] = "Hijo";          // nombre de la clase, SIN .class

        ProcessBuilder pb = new ProcessBuilder(sArg);
        pb.directory(new File("./bin"));   // carpeta donde esta
Hijo.class

        Process proc = null;
        try {
            proc = pb.start();              // lanza el proceso hijo
        } catch (IOException e) {
            e.printStackTrace();
            return;
        }

        System.out.println("PID hijo: " + proc.pid());

        // Leer la salida estandar del hijo
        try (BufferedReader br = new BufferedReader(
                new InputStreamReader(proc.getInputStream()))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Hijo

```
public class Hijo {
    public static void main(String[] args) {
        System.out.println("HOLA DAM");
        System.out.println("HOLA DAM-2");
    }
}
```

Variantes del comando (cambia solo la línea del ProcessBuilder)

```
ProcessBuilder pb = new ProcessBuilder("java", "Hijo");
// otra clase Java
ProcessBuilder pb = new ProcessBuilder("java", "Hijo", "5", "3");
// con argumentos -> llegan al args[] del hijo
ProcessBuilder pb = new ProcessBuilder("python3", "prueba.py");
// script Python (en ./bin)
ProcessBuilder pb = new ProcessBuilder("ls", "-la");
```

```
// comando del sistema
ProcessBuilder pb = new ProcessBuilder("firefox");
// aplicacion grafica
ProcessBuilder pb = new ProcessBuilder("bash", "-c", "ls -la | grep
txt"); // tuberias o comandos compuestos
```

Script Python de ejemplo (`prueba.py` en `./bin`):

```
import sys
print("HOLA DESDE Python")
sys.exit(0)
```

Leer argumentos en el hijo

Si el padre pasa `"java", "Hijo", "5", "3"`, en el hijo:

```
int a = Integer.parseInt(args[0]);   // 5
int b = Integer.parseInt(args[1]);   // 3
```

Ojo: el hijo debe estar compilado (`.class` en `./bin` si usas Eclipse). Si la ruta o el nombre está mal, el proceso arranca pero no imprime nada: mira `getErrorStream()` (sección 7).

3. Lanzar proceso con Runtime.exec [RT]

Alternativa más antigua a ProcessBuilder. No tiene `directory()`, `redirectInput()`, etc.: la carpeta de trabajo se pasa como tercer parámetro de `exec`. Devuelve el mismo `Process`, así que leer salida, `waitFor`, etc. funcionan igual.

```
import java.io.BufferedReader;
import java.io.File;
import java.io.IOException;
import java.io.InputStreamReader;

public class PadreRuntime {
    public static void main(String[] args) {
        String[] sArg = {"java", "Hijo"};
        // String[] sArg = {"python3", "prueba.py"};
        // String[] sArg = {"firefox"};

        Process proc = null;
        try {
            // exec(comando)                              -> el hijo debe
estar en la carpeta del proyecto
            // proc = Runtime.getRuntime().exec(sArg);
            // exec(comando, variablesEntorno, carpeta) -> equivale a
pb.directory()
            proc = Runtime.getRuntime().exec(sArg, null, new
File("./bin"));
        } catch (IOException e) {
            e.printStackTrace();
            return;
```

```
        }

        System.out.println("PID hijo: " + proc.pid());

        try (BufferedReader br = new BufferedReader(
                new InputStreamReader(proc.getInputStream()))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

	ProcessBuilder	Runtime.exec
Cómo se lanza	`new ProcessBuilder(cmd).start()`	`Runtime.getRuntime().exec(cmd)`
Carpeta de trabajo	`pb.directory(new File("./bin"))`	3er parámetro: `exec(cmd, null, new File("./bin"))`
Redirigir a ficheros	Sí (`redirectInput/Output/Error`)	No directamente
Reutilizable	Sí, puedes llamar `start()` varias veces	No

4. Esperar al hijo: waitFor y exitValue [WAIT]

- `proc.waitFor()` bloquea al padre hasta que el hijo termina y **devuelve el código de salida** (el número de `System.exit(n)` del hijo).
- `proc.waitFor(4, TimeUnit.SECONDS)` espera como máximo ese tiempo y devuelve un **boolean**: `true` si terminó, `false` si no.
- `proc.exitValue()` da el código de salida, pero **lanza `IllegalThreadStateException` si el hijo aún no ha terminado**. Úsalo solo si `waitFor(tiempo)` devolvió `true`.

Padre (las dos versiones)

```
import java.io.File;
import java.io.IOException;
import java.util.concurrent.TimeUnit;

public class PadreWait {
    public static void main(String[] args) {
        ProcessBuilder pb = new ProcessBuilder("java", "Hijo");
        pb.directory(new File("./bin"));
```

```
        Process proc = null;
        try {
            proc = pb.start();
        } catch (IOException e) {
            e.printStackTrace();
            return;
        }
        System.out.println("PID hijo: " + proc.pid());

        try {
            // ---- VERSION A: esperar SIN limite ----
            int codigo = proc.waitFor();
            System.out.println("Codigo de salida: " + codigo);

            // ---- VERSION B: esperar CON limite de tiempo ----
            // boolean terminado = proc.waitFor(4,
TimeUnit.SECONDS); // con 1 s -> false si el hijo tarda 3 s
            // if (terminado) {
            //     System.out.println("Codigo de salida: " +
proc.exitValue());
            // } else {
            //     System.out.println("El hijo sigue vivo, no ha
terminado a tiempo");
            //     proc.destroy();   // opcional: matarlo
            // }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Hijo (tarda 3 s y devuelve 10)

```
public class Hijo {
    public static void main(String[] args) {
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.exit(10);   // codigo de salida que recibe el padre (0
= todo bien)
    }
}
```

Prueba típica: con `waitFor(1, SECONDS)` sale `false`; con `waitFor(4, SECONDS)` sale `true` y el código 10.

Usar el código de salida para decidir

```
int codigo = proc.waitFor();
if (codigo == 0) {
    System.out.println("El hijo termino bien");
} else {
    System.out.println("El hijo fallo con codigo " + codigo);
}
```

5. Enviar datos del padre al hijo (entrada estándar) [STDIN]

El padre escribe en `proc.getOutputStream()` y el hijo lo lee con `Scanner(System.in)` como si alguien lo tecleara. Cada dato debe acabar en `\n` y hay que hacer `flush()` (o `close()`) para que se envíe de verdad.

Nombres confusos: desde el padre, `getOutputStream()` = la ENTRADA del hijo; `getInputStream()` = la SALIDA del hijo.

Padre

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;

public class PadreStdin {
    public static void main(String[] args) {
        ProcessBuilder pb = new ProcessBuilder("java", "Hijo");
        pb.directory(new File("./bin"));

        Process proc = null;
        try {
            proc = pb.start();
        } catch (IOException e) {
            e.printStackTrace();
            return;
        }
        System.out.println("PID hijo: " + proc.pid());

        // 1) ESCRIBIR en la entrada estandar del hijo
        try {
            BufferedWriter bw = new BufferedWriter(
                    new OutputStreamWriter(proc.getOutputStream()));
            bw.write("HOLA\n");      // cada dato con \n (el hijo usa
nextLine)
            // bw.write("5\n");
            // bw.write("3\n");
            bw.flush();               // IMPRESCINDIBLE para que
llegue
            bw.close();               // cierra la entrada: el hijo
```

```
recibe fin de datos
        } catch (IOException e) {
            e.printStackTrace();
        }

        // 2) LEER la salida estandar del hijo
        try (BufferedReader br = new BufferedReader(
                new InputStreamReader(proc.getInputStream()))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Hijo

```
import java.util.Scanner;

public class Hijo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String sInfo = sc.nextLine();          // se queda esperando
hasta recibir algo
        System.out.println("Recibido: " + sInfo);
        System.out.println("Te devuelvo el saludo");
        sc.close();
    }
}
```

Variante: mandar varios números y que el hijo los sume

Padre: `bw.write("5\n"); bw.write("3\n"); bw.flush();`

Hijo:

```
Scanner sc = new Scanner(System.in);
int a = Integer.parseInt(sc.nextLine());
int b = Integer.parseInt(sc.nextLine());
System.out.println(a + " + " + b + " es igual a " + (a + b));
sc.close();
```

Variante: leer hasta que no haya más datos

```
Scanner sc = new Scanner(System.in);
while (sc.hasNextLine()) {          // termina cuando el padre hace
bw.close()
    String linea = sc.nextLine();
    System.out.println("Procesado: " + linea.toUpperCase());
```

```
}
sc.close();
```

Variante: diálogo (enviar, leer respuesta, volver a enviar)

```
BufferedWriter bw = new BufferedWriter(new
OutputStreamWriter(proc.getOutputStream()));
BufferedReader br = new BufferedReader(new
InputStreamReader(proc.getInputStream()));
bw.write("dato1\n"); bw.flush();
String resp1 = br.readLine();       // el hijo debe responder con UNA
linea por dato
bw.write("dato2\n"); bw.flush();
String resp2 = br.readLine();
bw.close();
```

6. Redirigir entrada, salida y error a ficheros [REDIR]

Con `ProcessBuilder` el hijo puede leer de un fichero en vez del teclado y escribir en ficheros en vez de en pantalla. Las rutas relativas (`numero.txt`) se resuelven desde la **carpeta del proyecto** (donde se ejecuta el padre), no desde `directory()`.

Padre

```
import java.io.File;
import java.io.IOException;

public class PadreRedir {
    public static void main(String[] args) {
        ProcessBuilder pb = new ProcessBuilder("java", "Hijo");
        pb.directory(new File("./bin"));

        pb.redirectInput(new File("numero.txt"));   // el hijo LEE de
aqui (su System.in)
        pb.redirectOutput(new File("salida.txt"));  // lo que imprime
va aqui (sobrescribe)
        pb.redirectError(new File("error.txt"));    // los errores van
aqui

        Process proc = null;
        try {
            proc = pb.start();
            System.out.println("PID hijo: " + proc.pid());
            proc.waitFor();   // esperar para que salida.txt este
completo
            System.out.println("Terminado. Revisa salida.txt y
error.txt");
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Hijo (suma dos números leídos)

```
import java.util.Scanner;

public class Hijo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = Integer.parseInt(sc.nextLine());
        int b = Integer.parseInt(sc.nextLine());
        System.out.println(a + " + " + b + " es igual a " + (a + b));
        sc.close();
    }
}
```

`numero.txt` (en la carpeta del proyecto):

```
5
3
```

Resultado en `salida.txt`: `5 + 3 es igual a 8`

Probar error.txt

Lanza un comando con opciones falsas y su mensaje de error acaba en `error.txt`:

```
ProcessBuilder pb = new ProcessBuilder("ls", "-laeevefef");
pb.redirectError(new File("error.txt"));
```

Otras opciones de redirección

```
pb.redirectOutput(ProcessBuilder.Redirect.appendTo(new
File("salida.txt"))); // AÑADIR al final, no sobrescribir
pb.redirectErrorStream(true);              // juntar errores con la
salida normal (un solo stream)
pb.inheritIO();                            // el hijo usa la misma
consola que el padre (se ve todo directamente)
pb.redirectOutput(ProcessBuilder.Redirect.INHERIT); // solo la salida
a la consola del padre
```

7. Otros métodos de Process [METODOS]

Método	Qué hace	Devuelve
`pid()`	PID del proceso hijo	`long`
`isAlive()`	¿Sigue ejecutándose?	`boolean` (true = vivo)
`destroy()`	Pide al hijo que termine (como `kill`)	`void`
`destroyForcibly()`	Lo mata a la fuerza (como `kill -9`)	`Process`
`waitFor()`	Espera a que termine	`int` código de salida
`waitFor(t, unidad)`	Espera como máximo t	`boolean` terminó o no
`exitValue()`	Código de salida (excepción si	`int`

Método	Qué hace	Devuelve
	sigue vivo)	
`getInputStream()`	Salida estándar del hijo (lo que imprime)	`InputStream`
`getErrorStream()`	Salida de error del hijo	`InputStream`
`getOutputStream()`	Entrada estándar del hijo (lo que lee)	`OutputStream`
`info()`	Comando, usuario, hora de inicio…	`ProcessHandle.Info`

isAlive + destroy

```
Process proc = new ProcessBuilder("firefox").start();   // dentro de try/catch IOException
System.out.println("Vivo? " + proc.isAlive());   // true
Thread.sleep(5000);                                  // dentro de try/catch InterruptedException
proc.destroy();                                      // equivale a: kill <pid>   (kill -9 <pid> = destroyForcibly)
proc.waitFor();
System.out.println("Vivo? " + proc.isAlive());   // false
```

Desde el terminal: `ps -ef | grep java` para ver el proceso, `kill -9 <pid>` para matarlo a mano.

Leer los errores del hijo (getErrorStream)

```
try (BufferedReader brErr = new BufferedReader(
        new InputStreamReader(proc.getErrorStream()))) {
    String linea;
    while ((linea = brErr.readLine()) != null) {
        System.out.println("ERROR hijo: " + linea);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Información del proceso

```
System.out.println("PID padre: " + ProcessHandle.current().pid());
System.out.println("Comando hijo: " +
proc.info().command().orElse("?"));
System.out.println("Usuario: " + proc.info().user().orElse("?"));
```

8. Plantilla todo-en-uno y errores típicos [PLANTILLA]

Padre que lanza al hijo con argumentos, le envía datos, lee su salida y sus errores, y recoge el código de salida. Borra lo que no pida el ejercicio.

```
import java.io.*;
import java.util.concurrent.TimeUnit;
```

```java
public class PadrePlantilla {
    public static void main(String[] args) {
        ProcessBuilder pb = new ProcessBuilder("java", "Hijo",
"arg1");   // <- CAMBIAR comando/argumentos
        pb.directory(new File("./bin"));
// <- carpeta del .class

        try {
            Process proc = pb.start();
            System.out.println("PID hijo: " + proc.pid());

            // 1. Enviar datos al hijo (si el hijo usa Scanner)
            BufferedWriter bw = new BufferedWriter(new
OutputStreamWriter(proc.getOutputStream()));
            bw.write("dato\n");
// <- CAMBIAR datos
            bw.flush();
            bw.close();

            // 2. Leer salida del hijo
            BufferedReader br = new BufferedReader(new
InputStreamReader(proc.getInputStream()));
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println("HIJO: " + linea);
            }

            // 3. Leer errores del hijo
            BufferedReader brErr = new BufferedReader(new
InputStreamReader(proc.getErrorStream()));
            while ((linea = brErr.readLine()) != null) {
                System.out.println("ERROR: " + linea);
            }

            // 4. Esperar y recoger codigo de salida
            boolean terminado = proc.waitFor(5, TimeUnit.SECONDS);
            if (terminado) {
                System.out.println("Codigo de salida: " +
proc.exitValue());
            } else {
                System.out.println("No termino a tiempo, lo mato");
                proc.destroy();
            }
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Lanzar varios hijos a la vez

```
Process[] procs = new Process[3];
for (int i = 0; i < procs.length; i++) {
    ProcessBuilder pb = new ProcessBuilder("java", "Hijo",
String.valueOf(i));
    pb.directory(new File("./bin"));
    pb.redirectOutput(new File("salida" + i + ".txt"));   // cada uno
a su fichero
    procs[i] = pb.start();
}
for (Process p : procs) {
    System.out.println("PID " + p.pid() + " termino con " +
p.waitFor());
}
```

Errores típicos

Síntoma	Causa probable	Solución
No sale nada del hijo	Ruta o nombre de clase mal, o no compilado	Leer `getErrorStream()`; comprobar `./bin/Hijo.class`
`IOException: Cannot run program`	El ejecutable no existe en el PATH	Usar ruta completa (`which java`)
El hijo se queda colgado esperando	No se hizo `flush()`/`close()` o falta `\n`	Añadir `\n`, `flush()` y `close()`
`IllegalThreadStateException`	`exitValue()` con el hijo vivo	Llamar antes a `waitFor()`
`NullPointerException` en `proc.pid()`	`start()` falló y `proc` quedó `null`	Poner `return` en el catch o meter todo en el try
`salida.txt` vacío o incompleto	El padre no esperó al hijo	`proc.waitFor()` tras `start()`
Pasar `"ls -la"` en un solo String	ProcessBuilder lo trata como un único programa	Separar: `"ls", "-la"`